

안녕하세요, 지민수입니다.

1998.08.28
entersuk1998@naver.com / 01025955586

서비스 운영 관점에서 문제를 봅니다.

저는 보이는 기능보다 서비스가 실제로 운영될 때 어떤 문제가 생기는지를 먼저 봅니다. 업로드 상태, 조회 구조, 운영 지표처럼 사용자는 잘 느끼지 못하지만 서비스 품질을 좌우하는 기준은 결국 백엔드 구조에서 나온다고 생각합니다. 그래서 프로젝트에서도 기능 구현에 그치지 않고, 데이터 흐름과 상태 관리, 운영 기준까지 함께 정리하는 방식으로 문제를 풀어 왔습니다.

왜 막히는지 끝까지 확인하고 다시 고칩니다.

한 번에 잘 풀어내기보다, 원인을 먼저 확인하고 설명 가능한 방식으로 다시 정리하려고 합니다. 파워리프팅 3대 700과 국토종주 2회 등을 준비하며 익힌 반복과 복기의 습관도 개발에 그대로 가져오고 있으며, 이 태도가 결국 오래 버티는 구조를 만드는 데 가장 중요하다고 생각합니다.

협업과 도구 활용은 기준으로 통제합니다.

회의는 아젠다 기반으로 운영하고, ADR과 PR 규칙을 통해 작업 기준을 맞췄습니다. 프론트엔드와는 API 문서를 중심으로 빠르게 소통했고, AI도 단순 코드 생성 도구로 쓰기보다 문서와 규칙 안에서 통제 가능한 방식으로 활용했습니다.

희망 직무

백엔드 엔지니어

주요 기술

Java / Spring Boot / JPA
Redis / MySQL / AWS S3 / RDS
Docker Compose

경력

원티드랩 부트캠프
2025.09 - 2026. 03
백엔드 협업 과정 수료

팀 프로젝트 Shortudy 백엔드 개발 / 팀장

Link

<https://minji0828.github.io/minji-pop/#journey>

짧은 학습 영상을 업로드하고 소비하는 숏폼형 학습 플랫폼에서, 업로드·조회·운영 지표의 백엔드 구조를 담당했습니다.

프로젝트 개요

- 기간: 2025.11 - 2026.02
- 팀 규모 : 백엔드 4명 프론트엔드 2명
- 형태: 팀 프로젝트
- 서비스: 숏폼 학습 콘텐츠 업로드/조회 플랫폼

목표: 콘텐츠 품질을 통제하면서 운영 가능한 백엔드 구조 설계

내 담당 역할

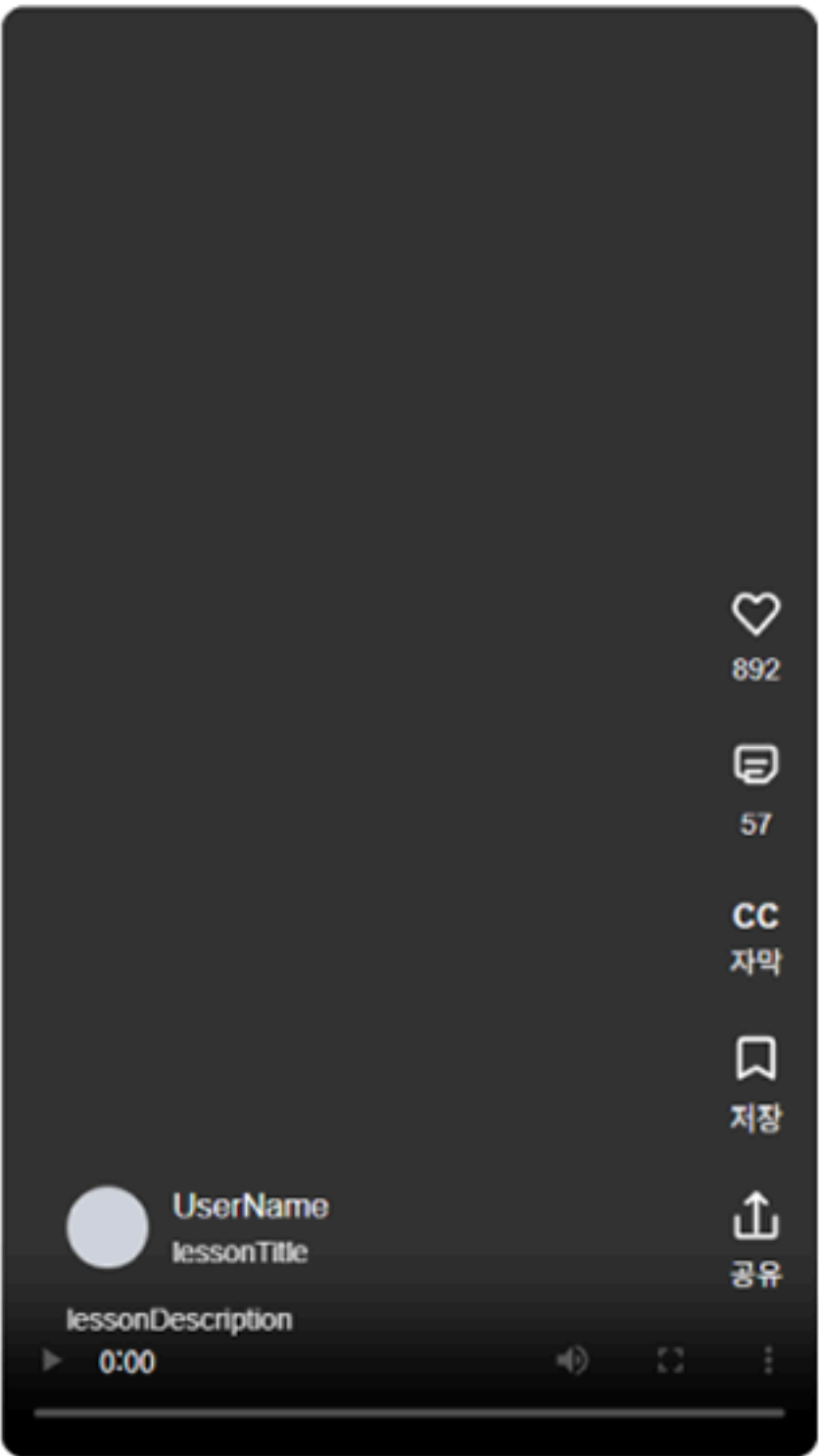
- S3 Direct Upload 기반 업로드 파이프라인 설계
- 업로드 상태 / 공개 상태 분리
- Redis write-back 기반 조회수 집계 구조 적용
- Projection 기반 조회 API 정리 및 읽기 경로 단순화
- BackOffice 운영 지표 API 구현
- 단일 EC2 구조의 한계를 확인하고 RDS 분리 방향으로 배포 구조 개선

핵심 기여 포인트

- 업로드를 단순 전송이 아니라 상태 관리 문제로 다룸
- 공개 조건과 검수 정책을 분리해 노출 기준 명확화
- 조회수 데이터 특성에 맞춰 적합성과 성능의 균형을 설계
- 운영 지표 계산 기준을 서버에서 고정해 숫자 해석 일관성 확보

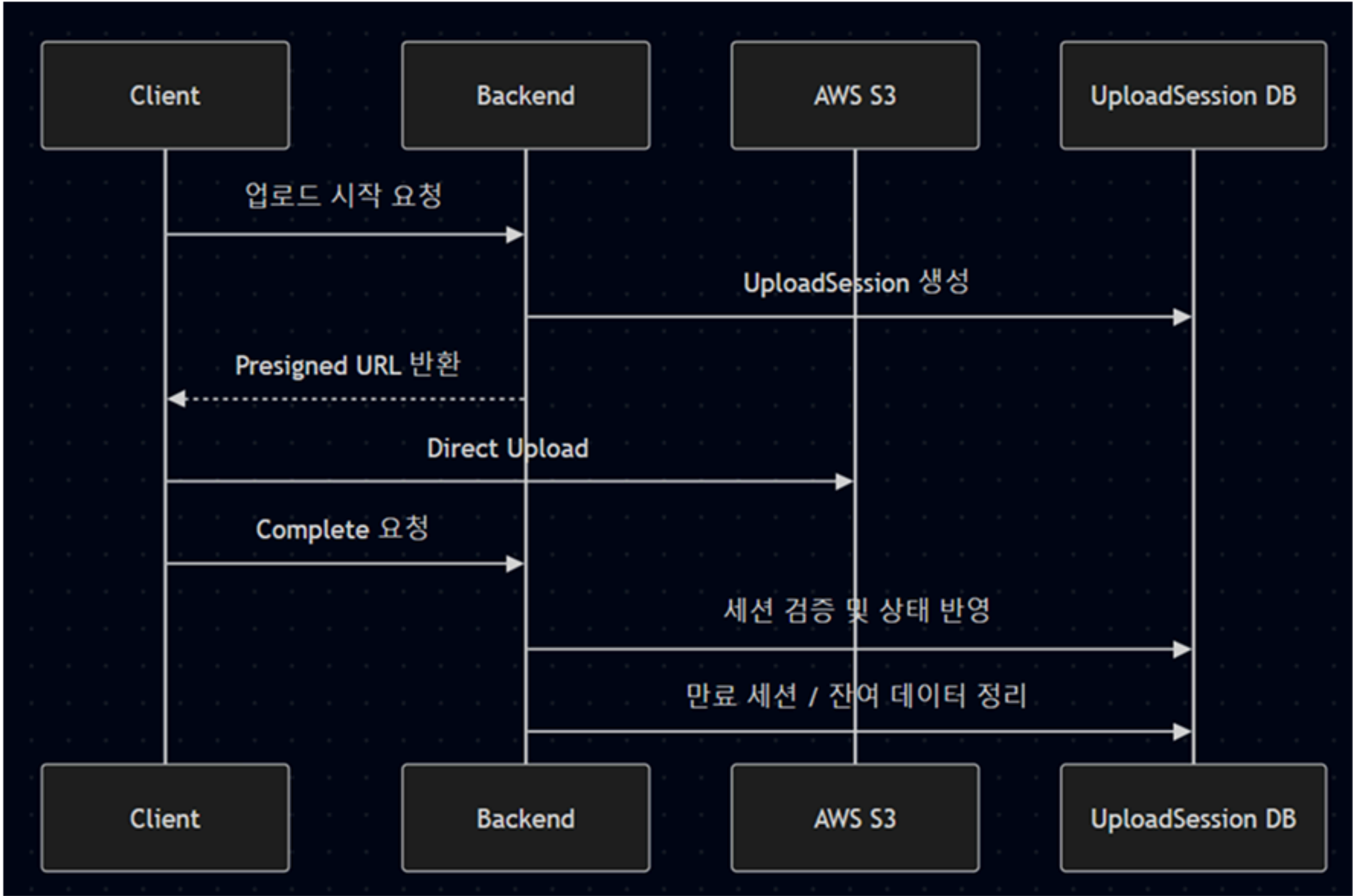
기능 추가보다, 서비스가 실제로 운영될 때 문제가 되는 지점을 먼저 정리하는 데 집중했습니다.

숏폼 학습 피드와 업로드 흐름을 포함한 사용자 서비스 메인 화면



S3 Direct Upload 기반 업로드 파이프라인 설계

서버 부하를 줄이면서도 업로드 중단, 재시도, 고아 데이터까지 관리할 수 있는 구조를 설계했습니다.



왜 이 방식을 선택했는가

- 사용자가 서버를 거치지 않고 S3에 직접 업로드하도록 구성
- 백엔드 부하를 줄이고 구조를 단순하게 가져가는 것이 우선순위였음
- 숏폼 서비스 특성상 초대용량 파일이 아니어서 Multipart Upload의 이점이 크지 않았음
- STS는 초기 프로젝트 단계에서 권한 설계와 클라이언트 구현 부담이 컸음
- 구현 속도와 구조 단순성을 기준으로 Presigned URL 기반 Direct Upload를 선택

내가 구현한 것

- Presigned URL 발급
- 업로드 세션 분리 및 상태 관리
- 재시도 시 이전 미완료 세션 정리
- 만료 세션 및 고아 데이터 정리 스케줄러 구성

설계 판단

- 업로드는 단순 파일 전송이 아니라 상태 관리 문제로 봤음
- 중간 이탈이나 실패가 발생해도 세션 기준으로 정리 가능하도록 UploadSession을 분리
- 요청 기반 정리만으로는 부족하다고 보고 스케줄러 기반 정리까지 포함

한계와 후속 개선

- 현재 구조는 클라이언트의 완료 요청을 일부 신뢰함
- 실제 S3 객체 존재 여부를 백엔드가 100% 보장하지는 못함
- 후속으로 S3 이벤트 기반 검증 파이프라인으로 보완 예정

조회수 데이터를 Redis write-back 구조로 재설계

조회수에는 모든 요청을 즉시 DB에 쓰기보다, 데이터 성격에 맞는 반영 전략이 더 중요했습니다.

```
}  
  
// Redis 조회수를 DB에 주기적으로 반영 (기본 1분) 식별 기준  
@Scheduled(fixedDelayString = "${shorts.view.flush-interval-ms:60000}")  
public void flushViewCounts() {  
    // 로그인: 사용자 ID  
    // 비로그인: IP + User-Agent 해시  
    shortsViewCountService.flushViewCounts();  
}
```

```
}  
  
// 중복 조회 방지를 위한 방문자 키 등록  
public boolean markUniqueView(Long shortId, String visitorId, Duration ttl) {  
    String key = UNIQUE_KEY_PREFIX + shortId + ":" + visitorId;  
    Boolean created = valueOperations.setIfAbsent(key, "1", ttl);  
    return Boolean.TRUE.equals(created);  
}
```

```
// 조회수 증가 처리  
@Transactional  
public void increaseViewCount(Long shortId, String visitorId) {  
    if (shortId == null || shortId <= 0) {  
        throw new BaseException(ErrorCode.INVALID_INPUT, "shortId: 값이 올바르지 않습니다");  
    }  
    if (visitorId == null || visitorId.isBlank()) {  
        throw new BaseException(ErrorCode.INVALID_INPUT, "visitorId: 값이 올바르지 않습니다");  
    }  
  
    boolean isNewView = viewCountRepository.markUniqueView(shortId, visitorId);  
    if (isNewView) {  
        viewCountRepository.increaseViewCount(shortId);  
    }  
}
```

```
// Redis 누적 조회수 DB 반영  
@Transactional  
public void flushViewCounts() {  
    Map<Long, Long> counts = viewCountRepository.findPendingViewCounts();  
    if (counts.isEmpty()) {  
        return;  
    }  
  
    for (Map.Entry<Long, Long> entry : counts.entrySet()) {  
        shortsRepository.updateViewCount(entry.getKey(), entry.getValue());  
    }  
    viewCountRepository.clearPending(counts.keySet());  
}
```



문제 상황

- 조회 요청마다 MySQL row를 직접 update하면 쓰기 경합이 커질 수 있었음
- 조회수는 결제 데이터처럼 즉시 정확성이 절대적인 값은 아니었음
- 따라서 매 요청마다 DB에 즉시 반영하는 구조는 비효율적이라고 판단

내가 구현한 것

- Redis INCR 기반 조회수 누적
- 주기적 flush를 통한 MySQL 반영
- 24시간 기준 unique view 정책 적용
- 로그인/비로그인 사용자 구분 집계

설계 판단

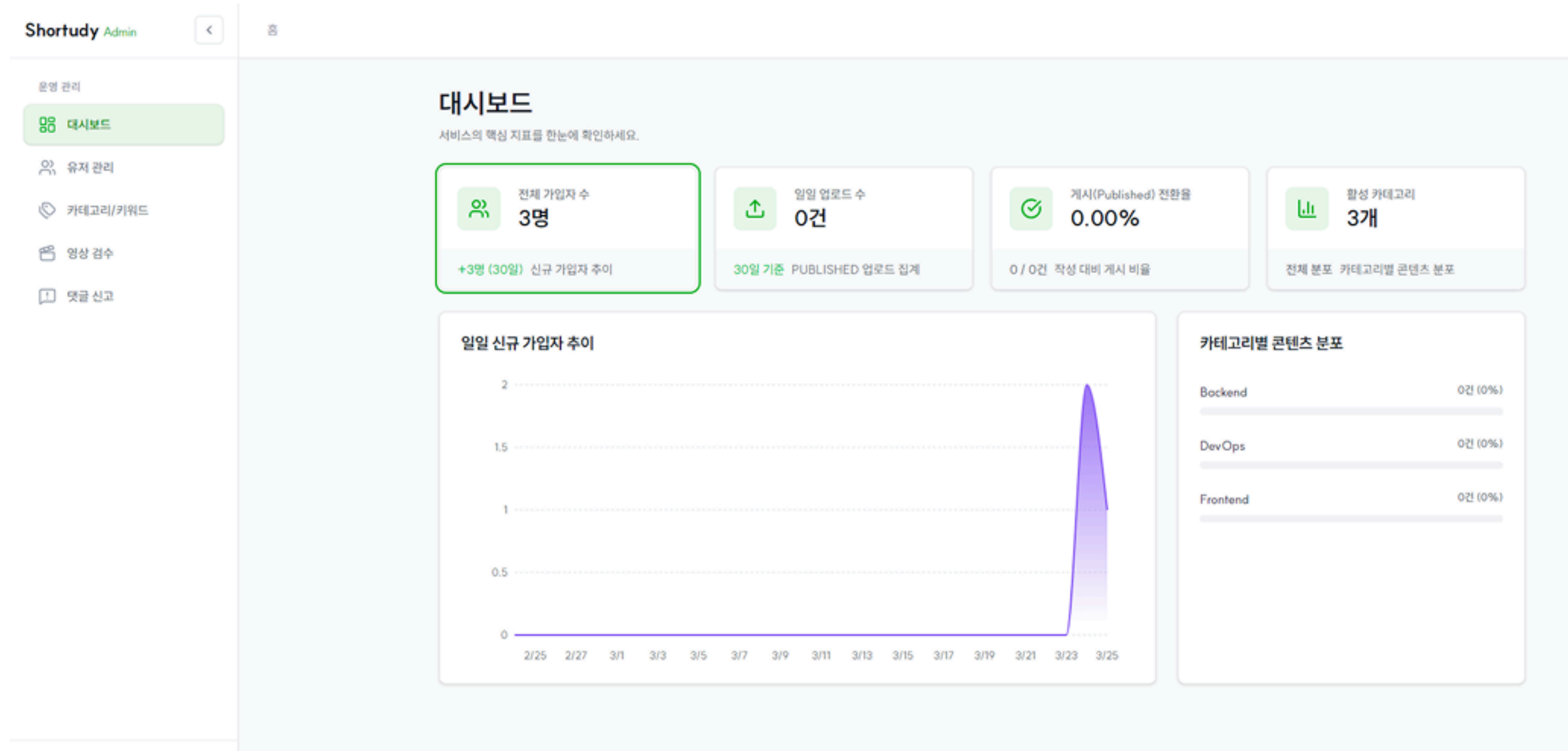
- 조회수는 데이터 성격에 맞는 집계 전략이 중요하다고 봤음
- 즉시 정확성보다 서비스 속도와 쓰기 부하 분산을 우선
- 그래서 Redis write-back 구조를 선택

한계와 후속 개선

- 비회원 식별에 IP + User-Agent 해시를 사용한 점은 한계가 있었음
- NAT 환경이나 정책 측면에서 더 신중한 접근이 필요했음
- 후속으로는 서버 발급 익명 UUID 쿠키 기반 식별이 더 적절

BackOffice 운영 지표 API 설계 및 기준 정리

운영 지표는 단순 조회가 아니라, 숫자를 해석하는 기준을 서버에서 고정하는 문제로 봤습니다.



문제 상황

- 운영 지표는 계산 기준이 조금만 달라도 숫자가 달라질 수 있었음
- 또한 프론트 공수가 제한적이어서, 운영 화면에 필요한 모든 지표를 한 번에 구현하기 어려웠음
- 그래서 현재 수집 가능하고 바로 활용할 수 있는 지표부터 우선순위를 나눠 단계적으로 제공할 필요가 있었음

내가 구현한 것

- BackOffice 운영 지표 API 구현
- 게시 추세 및 전환율 계산 로직 정리
- 기간 계산 기준과 입력 제한 조건 반영
- 클라이언트와 분리된 백오피스 구조 구성

설계 판단

- 기간 계산은 KST 기준으로 고정
- 전환율은 같은 기간에 생성된 전체 숫츠 대비 게시 완료 수 기준으로 계산
- 입력 기간 제한과 반올림 규칙까지 서버에서 명시

확장 방향

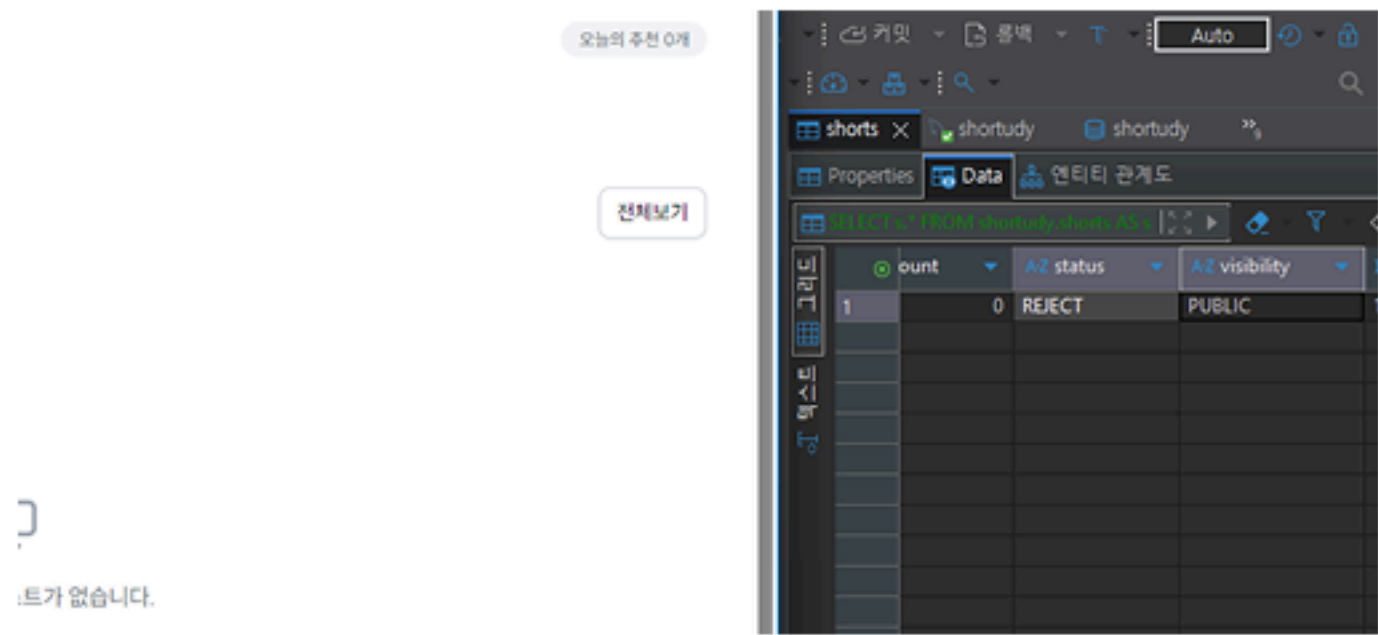
- 현재 수집 가능한 데이터부터 우선 제공
- 추후 백엔드 로그와 프론트 이벤트가 추가되면 더 세밀한 지표 확장 가능
- 즉, 운영 도구도 한 번에 완성하기보다 수집 가능한 정보부터 단계적으로 설계

- Phase 1: 현재 DB 기준으로 바로 수집 가능한 정보
- Phase 2: 백엔드 로그 추가 후 수집 가능한 정보
- Phase 3: 프론트 이벤트까지 포함해야 하는 정보

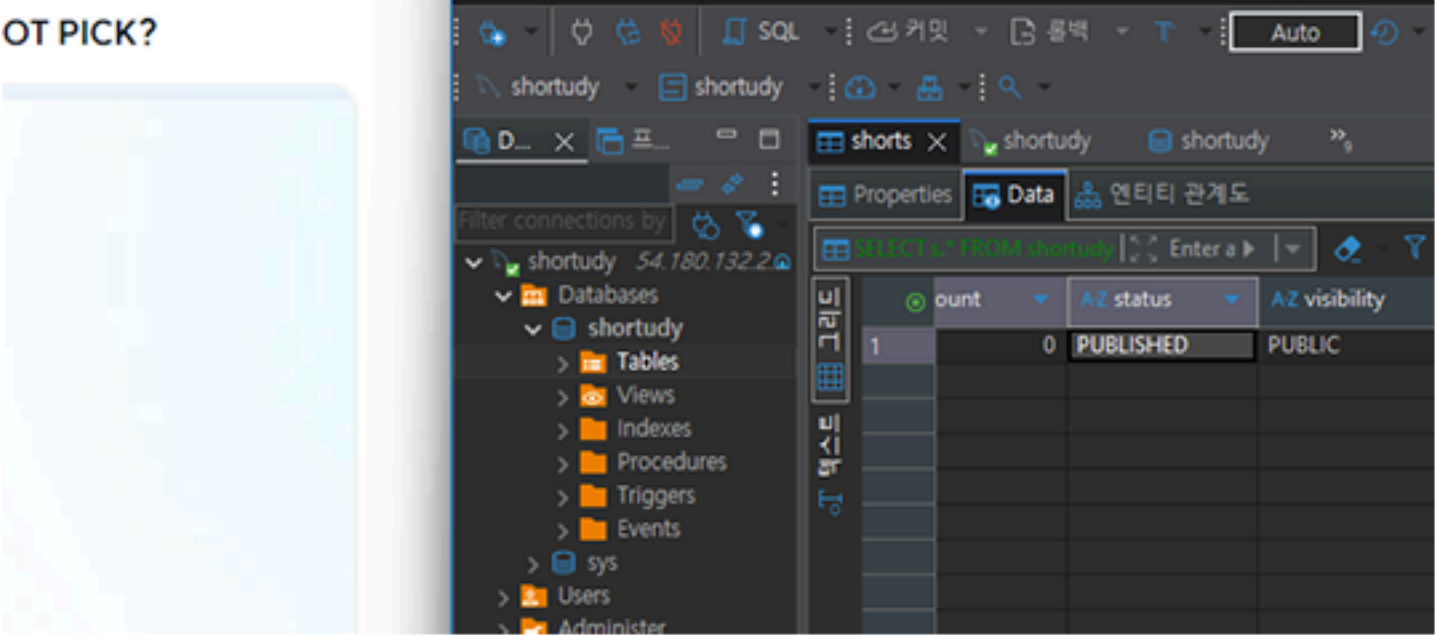
업로드 상태와 공개 상태 분리

업로드 완료가 곧 공개 가능 상태를 의미하지 않도록, 검수와 노출 정책을 별도 상태로 관리했습니다.

REJECT, PUBLIC 시 안보임



PUBLISHED, PUBLIC 시 보임



업로드 완료 != 공개 가능

영상 검수

자동 AI 검수 결과를 확인하고 게시/반려를 결정합니다.

검수 대기

1건

AI 점검

0건

게시

0건

반려

0건

필터

전체 상태

검수 요청 목록

소스ID	제목	영상	작성자	상태	상태 설명	AI 결과	검수사유	등록 시각	액션
14	테스트	영상 보기	지민수	검수 대기	설명 보기	검수 대기	AI 검수 완료 후 확인 가능	3.25. 오전 07:38	게시 반려

총 1건

이전

1 / 1

다음

노출 조건

- Upload Status = PUBLISHED
- Visibility = PUBLIC
- 두 조건을 모두 만족할 때만 사용자에게 노출

업로드 완료

- > 검수 통과
- > 공개 설정 확인
- > 사용자 노출

문제 상황

- 이 서비스에서는 사용자가 공개를 선택했다고 해서 업로드 직후 바로 노출되면 안 됨
- AI 검수 결과와 운영 정책이 함께 반영되어야 실제 공개 여부가 결정됐음
- 업로드 상태와 공개 상태가 하나의 값에 섞이면 정책이 불명확해질 수 있었음

내가 적용한 구조

- 업로드 상태와 게시 가능 상태를 분리
- 사용자의 공개 설정과 서비스 검수 상태를 별도로 관리
- 조회 시 두 조건을 모두 만족하는 경우에만 노출

설계 판단

- 업로드 성공은 파일 전송 완료일 뿐, 공개 가능 상태와는 다름
- 사용자 의사와 서비스 정책을 하나의 상태로 합치지 않기로 판단
- 정책을 서비스 로직 곳곳이 아니라 상태와 조회 조건에 명시적으로 반영

정책이 복잡할수록 상태를 분리해야 변경 지점이 명확해진다고 봤습니다.

Projection 기반 조회 API 및 읽기 경로 정리

복잡한 패턴보다, 필요한 데이터를 한 번에 가져오고 응답 조립 책임을 단순하게 나누는 데 집중했습니다.

초기 접근

Facade 중심으로 조회 책임을 감싸는
방향 검토

```
@Transactional(readOnly = true)
public ShortsResponse getShortsDetails(Long shortsId, Long userId) {
    // 1. 핵심 데이터 조회
    Shorts shorts = shortsService.findShortsWithDetails(shortsId);

    // 2. 부가 데이터 병렬/개별 조회 (댓글, 좋아요 등)
    long commentCount = commentCountProvider.commentCount(shortsId);
    boolean isLiked = (userId != null) &&
        shortsLikeRepository.existsByUserIdAndShortsId(userId, shortsId);

    // 3. Redis 실시간 조회수 보정 (Write-Back 반영)
    long realTimeViewCount = calculateRealTimeViewCount(shorts, shortsId);

    // 4. 단일 응답 객체 조립
    return ShortsResponse.of(shorts, commentCount, realTimeViewCount, isLiked);
}
```



최종 접근

Projection 조회 + 배치 조회 + Redis
병합으로 읽기 경로 단순화

```
[전체 목록 조회 쿼리]
JQuery<ShortsResponse> findResponsesByStatus(@Param("status") ShortsStatus status, @Param("userId") Long us
    "SELECT new com.example.shortstudy.domain.shortcuts.dto.ShortsResponse(" +
    "s.id, s.title, s.description, s.videoUrl, s.thumbnailUrl, s.durationSec, s.status, s.visibility, " +
    "u.id, u.nickname, u.profileUrl, " +
    "c.id, c.name, " +
    "null, " +
    "s.viewCount, s.likeCount, " +
    "(SELECT count(cm) FROM Comment cm WHERE cm.shortcuts = s), " +
    "s.createdAt, s.updatedAt, " +
    "(SELECT count(1) > 0 FROM ShortsLike l WHERE l.shortcuts = s AND l.user.id = :userId)) " +
    "FROM Shorts s " +
    "JOIN s.user u " +
    "JOIN s.category c " +
    "WHERE s.status = :status AND s.visibility = 'PUBLIC'"
    )
```

문제 상황

- 쏫츠 목록 응답에는 기본 정보 외에도 키워드와 실시간 조회수를 함께 조합해야 했음
- 엔티티 그래프 중심으로 풀면 조회 비용과 응답 조립이 복잡해질 수 있었음
- 초기에는 Facade처럼 묶는 방향도 고민했지만 구조가 더 무거워질 수 있었음

내가 적용한 구조

- Projection 기반 기본 조회
- 키워드 배치 조회
- Redis pending 값 병합
- Query Service에서 읽기 전용 책임 분리

설계 판단

- 핵심은 패턴 이름보다 필요한 데이터를 적절히 조회하는 구조였음
- N+1을 줄이면서도 응답 조립 책임을 과하게 복잡하게 만들지 않는 방향을 선택
- 결과적으로 읽기 경로를 더 단순하고 명확하게 정리